Neural Network Code Report

Time-Aware Reinforcement Learning for Ball Catching under Visual Delay

ME5418 Group 30

1 Code Explanation

The networks are the policies that PPO will train on our ball-catching environment. There is one actor-critic network per PPO variant of the proposal. Each maps the observation to a Gaussian over the 2-D action (net acceleration) and to a value estimate $V(s)$ for the critic. The code is a new package, `networks/` (about 470 lines), with a demo and 20 unit tests; the whole repository now has 141 tests.

1.1 Organisation

File	What it does
`policies.py`	The three networks, a common `ActorCritic` interface (`act`, `evaluate`), `make_policy(variant)` and `env_config_for(variant)`.
`ppo.py`	What PPO needs around the networks: `collect_rollout`, advantages (`compute_gae`) and the clipped loss (`ppo_loss`).
`demo.py`	Builds each network and shows a forward and a backward pass on environment data.
`tests/`	Shapes, memory, memory reset, agreement of step-wise and sequence evaluation, gradients in every layer, GAE and loss values, and the demo script.

1.2 The three networks

Variant	Network	Inputs	Parameters	Role in the study
`single_frame`	MLP, 2×64 tanh, separate actor and critic	9	9,797	Reactive baseline: no memory
`recurrent`	encoder $64 \to$ GRU $64 \to$ actor and critic heads	9	34,117	Memory, no timestamps
`temporal`	same as `recurrent`	11	34,245	Memory plus capture interval and age

The two recurrent variants differ only in the two timing inputs (128 extra weights), so a difference in catching can be attributed to timing information rather than to capacity. The **actor** outputs the mean of the Gaussian; its standard deviation is a learned parameter per action axis, independent of the observation, initialised to $e^{-0.5} = 0.61$ so that early actions explore most of $[-1, 1]$. Linear layers use orthogonal initialisation; the GRU uses PyTorch's default initialisation. The last actor layer has a small gain (0.01), so an untrained network starts with mean actions near zero.

1.3 Memory across a throw

The GRU is updated once per control step (60 Hz), not once per camera frame, so between frames it still sees the net state, the new-frame flag and, in the temporal variant, the growing observation age. All tensors are sequences `[T, B, inputs]`; a flag `episode_start` marks the first step of each throw, where the memory is reset ($h \leftarrow 0$), so one sequence can hold several throws without one ball leaking into the next. Two methods use the same forward pass: `act` chooses one step's action during a rollout (no gradients), and `evaluate` re-runs a whole stored sequence with gradients for the update. A test checks that both give identical log-probabilities, a common source of silent bugs in recurrent PPO.

1.4 Forward and backward pass

`collect_rollout` plays the network in the environment and stores observations, sampled actions, their log-probabilities, values, rewards and throw boundaries. `compute_gae` turns these into advantages ($\gamma = 0.99$, $\lambda = 0.95$; no bootstrapping across the end of a throw). `ppo_loss` is the clipped objective of PPO, with $r = \pi_{\text{new}}(a|s)/\pi_{\text{old}}(a|s)$ and normalised advantages $\hat{A}$:

$$L = -\,\mathbb{E}\big[\min\big(r\hat{A},\ \text{clip}(r, 0.8, 1.2)\,\hat{A}\big)\big] + 0.5\,\mathbb{E}\big[(V - R)^2\big] - c_e\,\mathbb{E}[\text{entropy}].$$

Excerpt of `python -m networks.demo --variant temporal` (untrained network, jittered delay):

```
input size 11, 34,245 trainable parameters
Forward pass: 128 control steps, 2 throws finished, 0 caught (untrained network)
  t=5  obs [-0.18 -0.04 +0.04 +1.00 +1.00 -0.00 +0.00 -0.04 +0.00 +0.00 +0.32]
       -> mean [-0.00 -0.00] | action [-0.08 +0.26] | value +0.111
Backward pass: loss 0.0367 = policy -0.0000 + 0.5 x value 0.0734   (entropy 1.838)
  gradient norm per part: log_std 0.1223, encoder 0.1905, gru 0.4812, actor 0.4376, critic 0.3692
  after one Adam step (lr 0.0003): loss 0.0367 -> 0.0262
```

At $t = 5$ the first camera frame arrives (new-frame flag 1, age 64 ms); before that the network sees only the net state. The policy term is zero before an update because old and new policies are equal; every part of the network receives a gradient, and one optimiser step lowers the loss.

How to run.

```
conda env create -f environment.yml && conda activate ball-catch
python -m networks.demo                        # all three variants
python -m networks.demo --variant temporal     # one variant (single_frame, recurrent, temporal)
python -m networks.demo --help                 # rollout length, seed, delay, learning rate
python -m unittest                             # all 141 tests
```

2 Existing Codes and Libraries

- **PyTorch 2.5.1** (added to `environment.yml`): layers (`nn.Linear`, `nn.GRUCell`), the `Normal` distribution for log-probabilities and entropy, automatic differentiation and the Adam optimiser. The networks run on a CPU; on Linux pip installs the CUDA build, which also works without a GPU.
- **Our Gym environment** (previous milestone): `env_config_for(variant)` switches its timing inputs on or off, so every network gets an observation of the right size on the same throws.
- **Method references**: the clipped PPO objective (Schulman et al., 2017) and generalised advantage estimation (Schulman et al., 2016). Initialisation and loss weights follow common single-file PPO implementations such as CleanRL.

Why not Stable-Baselines3? Its recurrent PPO (in `sb3-contrib`) uses an LSTM with its own network layout. Our study needs two recurrent networks that are identical except for two inputs, a memory reset exactly at each throw, and the ability to inspect every tensor. Writing the networks ourselves (about 470 lines) gives that control and lets us test each piece in isolation. The environment also runs on Google Colab, which ships NumPy 2: patching one line of Gym's environment checker is enough (README).

3 Reflections and Lessons Learned

Changes from the proposal. The three main variants are built as proposed; history stacking and auxiliary prediction stay optional extensions. Decisions the proposal left open:

- **Shared memory for actor and critic.** One encoder and one GRU feed both heads, so a single hidden state is carried through a throw. The single-frame MLP keeps separate actor and critic.
- **Unbounded Gaussian.** Actions are sampled without squashing and clipped to $[-1, 1]$ by the environment; PPO uses the unclipped sample. If training pushes against the limits, a tanh squashing is the fallback.
- **Capacity.** Hidden size 64 everywhere. The two recurrent policies are matched; the MLP is smaller (9.8k vs 34k parameters), which matters less because the core comparison is between the two recurrent policies.

Lessons learned.

- **Test the network on the real environment, not only on random tensors.** The demo showed that the network sees no ball for the first five steps of a throw, which random inputs hide.
- **Recurrent PPO has two code paths.** Acting step by step and re-evaluating a sequence must agree exactly, including memory resets; we test this directly.
- **Simple sanity checks catch bugs early**: the policy loss is zero before an update, gradients reach every layer, and one optimiser step lowers the loss.

Next steps. Write the training loop (several epochs per rollout, minibatches of whole throws, parallel environments); raise γ to 0.995–0.999, since the reward comes at the end of a 36–72 step throw and $0.99^{72} \approx 0.49$; tune the control-cost weight λ on held-out seeds; normalise the timing inputs if needed; implement the predictive baseline and compare all policies.

AI tool declaration. We used Claude and GPT, through the AI assistant, to help design, write and test the network code, and to draft and optimise this report. The requirements for the project were proposed by us after discussion. We are responsible for the content and quality of the submitted work.